Knowledge Graphs

From Triples to Retrieval

A practical tutorial with a companion lab

Dr. Anish Roychowdhury | Generative AI Architect & Educator

October 2026

What This Note Covers

A knowledge graph stores facts about entities as typed statements, adds a schema that says which statements are allowed, and supplies the machinery to build, check, query and learn from the result. This note teaches that whole chain with one running example and one dataset, so that every claim can be reproduced.

The example is a fictional consultancy, *Meridian Analytics*, with 241 entities (people, departments, projects, technologies, clients and cities) and 1,332 facts. All data are synthetic and are generated by a script with a fixed seed. Three deliverables go together:

- this **note**, which explains the ideas and reports measured results;
- the **data files**, eleven CSV and JSON files that include deliberate noise, errors and ambiguity;
- a **Jupyter notebook**, organised as ten labs, that runs every method on those files and ends each lab with exercises.

Sections	Content
1–2	Why graphs, and the two main data models: RDF triples and property graphs.
3	Ontologies, constraints and inference.
4	Querying with SPARQL, Cypher and a small path language.
5	Building the graph: entity resolution, text extraction, validation.
6	Analysis: centrality, communities, impact analysis and rule mining.
7	Embeddings and link prediction with TransE and DistMult.
8	Knowledge graphs and language models: retrieval, evidence and failure modes.
9–11	Engineering practice, a map from this note to the lab, and exercises.

The reader is assumed to know basic Python and relational databases. No prior exposure to semantic web standards is needed; the relevant parts of RDF, SPARQL, RDFS and SHACL are introduced where they are used.

Contents

1 Why Knowledge Graphs **4**
- 1.1 The problem . . . 4
- 1.2 Definition and ingredients . . . 4
- 1.3 When a graph is the wrong tool . . . 4
- 1.4 The running example . . . 5

2 Data Models **6**
- 2.1 Triples and RDF . . . 6
- 2.2 Property graphs . . . 7
- 2.3 Choosing between them . . . 7
- 2.4 What the lab uses . . . 7

3 Schema, Constraints and Inference **8**
- 3.1 Ontologies . . . 8
- 3.2 RDFS, OWL and SHACL . . . 9
- 3.3 Forward chaining . . . 10
- 3.4 Constraints as executable schema . . . 10

4 Querying **10**
- 4.1 SPARQL . . . 10
- 4.2 Paths as the unit of multi-hop questions . . . 11
- 4.3 Why traversal is cheap . . . 11

5 Building the Graph **12**
- 5.1 Entity resolution . . . 12
- 5.2 From text to triples . . . 13
- 5.3 Validating against the schema . . . 15

6 Analysing the Graph **15**
- 6.1 Centrality . . . 15
- 6.2 Communities . . . 16
- 6.3 Impact analysis . . . 16
- 6.4 Rule mining . . . 17

7 Embeddings and Link Prediction **18**
- 7.1 Two classic models . . . 18
- 7.2 Evaluation protocol . . . 19
- 7.3 Results . . . 19
- 7.4 Which relations are learnable . . . 20
- 7.5 What the space looks like . . . 20

8 Knowledge Graphs and Language Models **21**
- 8.1 Where retrieval falls short . . . 21
- 8.2 The experiment . . . 21
- 8.3 Results . . . 22
- 8.4 Evidence-guided context . . . 23
- 8.5 Failure modes . . . 23

9 Engineering Practice **24**
- 9.1 Decisions to make early . . . 24
- 9.2 Quality . . . 24
- 9.3 Common pitfalls . . . 24
- 9.4 People and privacy . . . 24
- 9.5 A staged way to start . . . 25

10 Using the Lab Notebook **25**
10.1 Map from the note to the labs 25
10.2 Running it 25
10.3 Why the data are synthetic 26
11 Exercises **26**
11.1 Concepts 26
11.2 Lab extensions 27

1 Why Knowledge Graphs

1.1 The problem

Most organisations record the same facts in several places. An employee appears in an HR system, an issue tracker, a wiki and a staff directory. A client is a row in a CRM, a line on an invoice and a party to a contract. Each system describes its own slice with its own identifiers, and the questions that matter cut across all of them. Which clients are served by projects that use a given library? Who could cover for a colleague on a project? What is affected if a component is retired?

Relational databases answer such questions, but at a price. Every new kind of relationship needs a new table and new join logic, and a question whose depth is not known in advance ("everyone above this person in the reporting chain") needs recursive queries that are awkward to write and slow to run. A knowledge graph takes the opposite approach: the relationships are the data, and the schema is allowed to grow with them.

The phrase that popularised the idea outside research was "things, not strings" [1]. A search for a name should return the entity the name refers to, together with what is known about it, and not only the documents that contain the characters. Public graphs built on this principle include DBpedia, Freebase, YAGO and Wikidata [2, 3, 4, 5], and large companies run internal graphs for search, recommendation and compliance [6].

1.2 Definition and ingredients

Hogan and colleagues define a knowledge graph as a graph of data intended to accumulate and convey knowledge of the real world, with nodes for entities of interest and edges for the relations between them [7]. In practice three ingredients are always present.

Data.
Statements of the form *(head, relation, tail)*, called triples. For example, (Hiroshi Bhat, `leads`, Project Atlas).

Schema.
An ontology that names the kinds of entity and relation and says which statements make sense. It lets software check the data and derive new statements from it.

Processes.
The pipeline that builds the graph from sources, keeps it correct, and serves queries, analytics and learning. Without these the graph is a one-off diagram.

This note is organised around the three ingredients: data models in Section 2, schema in Section 3, and the processes in Sections 4 to 8.

Key Idea

A knowledge graph is not a particular technology. It is a way of organising facts so that entities have stable identities, relations are typed, and the schema is data that programs can read.

1.3 When a graph is the wrong tool

A graph is not always the right choice. If the task is to aggregate uniform records (monthly revenue by region), a columnar warehouse is faster and simpler. If the schema is small and stable and the questions need joins only one or two tables deep, a relational database is enough. A graph earns its cost when several of the following hold:

- entities of many types are connected, and the connections are what users ask about;
- the depth of a traversal is not known in advance;
- facts come from several sources that must be reconciled;
- the schema changes often, or new relation types appear as the work grows;
- answers must be explained, so that provenance of each fact matters.

1.4 The running example

All examples use *Meridian Analytics*, a fictional technology consultancy. The data were generated so that they look like what organisations really hold: structured records, free-text documents, duplicate entries across systems and a certain amount of error. Figure 1 shows the schema. Table 1 lists the files.

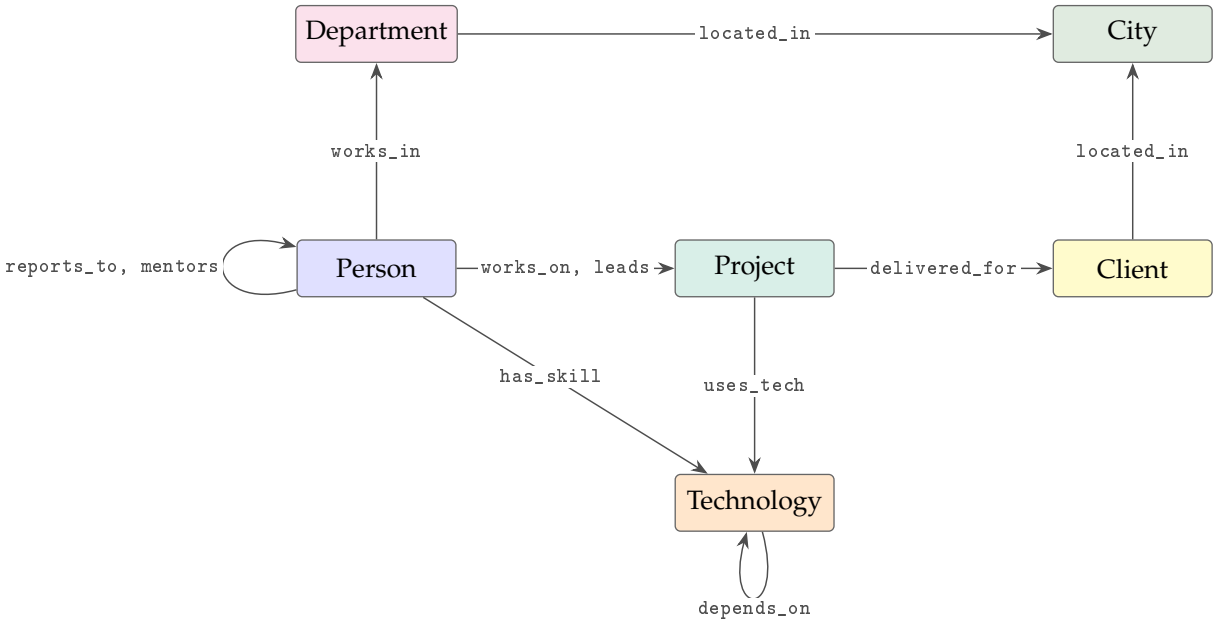


Figure 1: Schema of the Meridian graph. Boxes are classes and arrows are relations. `Person` has the subclasses Executive, Manager, Engineer, Analyst and Researcher; `Technology` has the subclasses Language, Database, Framework, CloudService and MLTool.

File	Content	Rows
`entities.csv`	nodes with id, label, type and attributes	241
`triples.csv`	edges with source, confidence and year	1,332
`ontology.json`	class hierarchy and property constraints	1
`aliases.csv`	alternative names; ambiguity flags	261
`gazetteer.csv`	curated dictionary for mention detection	434
`documents.csv`, `doc_sentences.csv`	document metadata and 739 sentences	160 docs
`doc_gold_triples.csv`	answer key for text extraction	859
`raw_records.csv`	noisy person and client records from 7 systems	374
`triples_dirty.csv`	the clean edges plus 90 injected errors	1,422
`questions.csv`	multi-hop questions with gold answers	60

Table 1: Data files. Every file is reproducible from `scripts/generate_data.py` with seed 2026.

The graph has 120 people in five roles, 9 departments, 45 projects, 37 technologies in five classes, 24 clients and 6 cities, linked by 10 relations. The generator plants structure on purpose: departments favour certain technologies and client industries, people tend to hold skills in

the technologies of their own projects, and about a third of the collaboration edges cross department lines. These regularities are what the analysis and learning methods later discover.

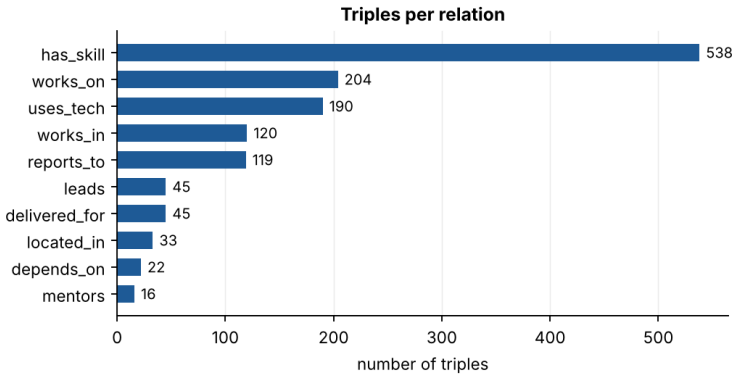


Figure 2: Number of triples per relation. The skills survey dominates the graph, which matters later because its confidence is the lowest (mean 0.77).

2 Data Models

Two models dominate practice. RDF stores a graph as a set of triples with globally unique identifiers. A property graph stores nodes and edges that each carry key-value properties. They describe the same real-world structure and differ in what is easy to say.

2.1 Triples and RDF

The Resource Description Framework represents every fact as a triple of subject, predicate and object [8]. Subjects and predicates are identified by URIs, and objects are either URIs or literal values such as strings and numbers. Because the identifiers are global, two graphs that use the same URI for the same thing can be merged by taking the union of their triples. This was the central promise of the semantic web [9].

A fragment of the Meridian graph in Turtle syntax:

```
@prefix ex:   <http://meridian.example/kg/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:prj_01  a             ex:Project ;
           rdfs:label    "Project Atlas" ;
           ex:uses_tech  ex:tec_08 , ex:tec_13 ;
           ex:delivered_for  ex:cli_08 .
ex:per_096 a             ex:Manager ;
           rdfs:label    "Hiroshi Bhat" ;
           ex:leads      ex:prj_01 .
```

The keyword `a` abbreviates `rdf:type`. A semicolon repeats the subject and a comma repeats subject and predicate. Everything, including the schema, is a triple, which is why a single query language and a single reasoner can work on data and schema together.

RDF has two well-known limits. First, a triple has no slot for properties of the statement itself, such as the confidence or source of a fact. The classical workarounds are reification (turn the statement into a node), named graphs (put statements in groups that carry metadata) and, more recently, RDF-star, which lets a triple be the subject of another triple. Second, a relation is a plain edge: there is no built-in notion of an ordered list or a typed, attributed relationship.

2.2 Property graphs

In a property graph, nodes and edges have labels and arbitrary key-value properties, and an edge has its own identity [10]. The same fact as above, with its provenance, becomes natural to store:

```
(:Manager {id:'per_096', name:'Hiroshi Bhat'})
   -[:LEADS {source:'Project_Registry', confidence:0.91, valid_from:2020}]->
(:Project {id:'prj_01', name:'Project Atlas'})
```

Cypher, the most widely used property-graph query language, writes patterns in the same ASCII-art style [11]. The question "for which clients has Priya Nair worked?" reads:

```
MATCH (p:Person {name: 'Priya Nair'})-[:WORKS_ON]->(:Project)-[:DELIVERED_FOR]->(c:Client)
RETURN DISTINCT c.name
```

2.3 Choosing between them

	RDF	**Property graph**
Unit of data	triple; every node and relation has a URI	node and edge with labels and properties
Edge properties	reification, named graphs or RDF-star	native
Schema language	RDFS, OWL, SHACL (W3C standards)	vendor constraints; no common reasoning standard
Query language	SPARQL (standard)	Cypher, Gremlin; the GQL standard is converging
Strength	integration across organisations, inference	application development, path queries, tooling
Typical use	open data, ontologies, linked data	enterprise applications, recommendation, fraud

Table 2: Comparison of the two models. The boundary is blurring: many RDF stores support RDF-star and many property graph systems can import RDF.

Practical Tip

The choice matters less than the discipline around it. Keep identifiers stable, record provenance on every edge, and write the schema down. Converting between the two models is routine when those three things are in place.

2.4 What the lab uses

The lab keeps the graph in two simple tables, an entity table and an edge table, and builds the working structures from them in memory. A NetworkX `MultiDiGraph` serves as the property graph: nodes carry label and type, and edges carry the relation name as key together with source, confidence and year. An `rdflib` graph serves as the RDF view; it holds types, labels and relation triples and leaves the edge properties out. Keeping the tables as the source of truth and deriving both models from them makes the comparison honest, since they contain the same statements.

The degree distribution of the property graph is strongly uneven (Figure 3). The median node has 9 edges, and the most connected node, the language Python, has 52. Such hubs matter for

retrieval: a query that reaches a hub returns a large and mostly irrelevant neighbourhood, a point that returns in Section 8.

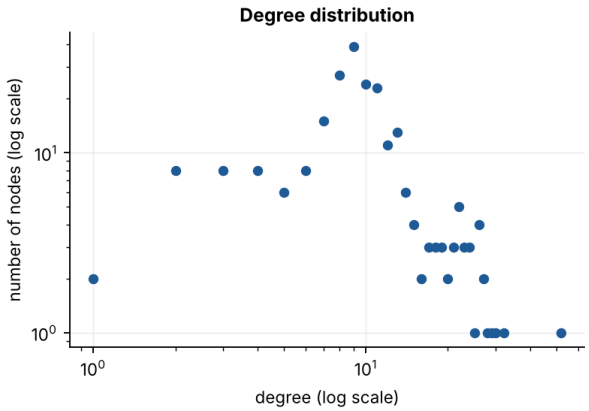


Figure 3: Degree distribution on logarithmic axes. Most nodes have few edges; a handful of technologies, such as Python, have many.

A graph is easier to understand locally than globally. Figure 4 shows every node within two hops of Project Atlas after removing the dense skill, reporting and mentoring edges.

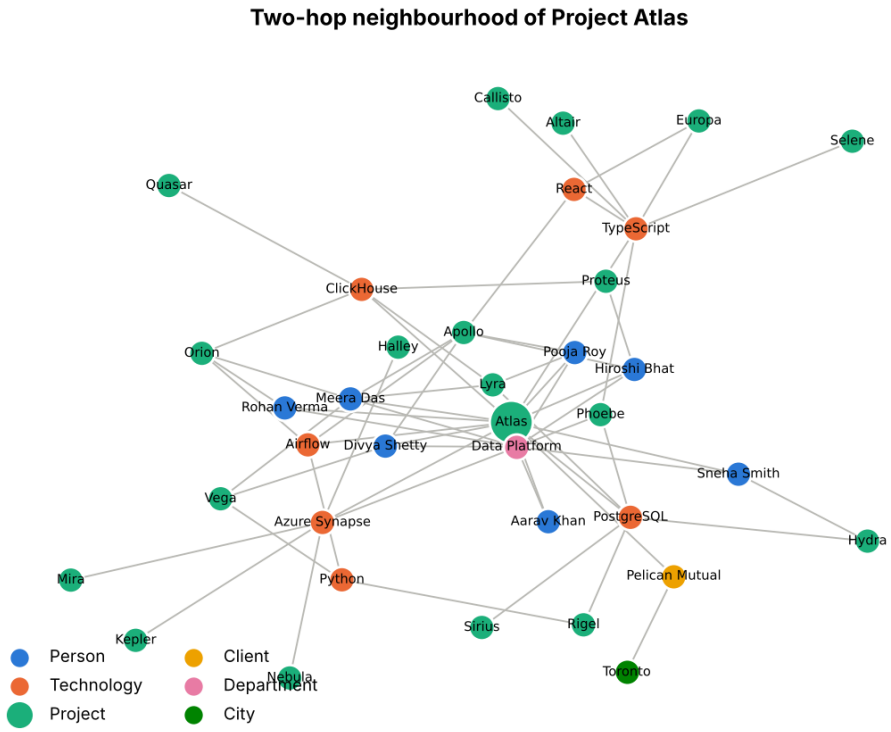


Figure 4: Two-hop neighbourhood of Project Atlas: its lead and team, its five technologies, its client and the client's city, plus the other projects that share a person or a technology.

3 Schema, Constraints and Inference

3.1 Ontologies

Gruber's often cited definition calls an ontology an explicit specification of a conceptualisation [12]. In a knowledge graph this means a vocabulary of classes and relations, together with

statements about them. The Meridian ontology has 16 classes in two hierarchies and 10 properties.

Each property has a *domain*, the class its subjects belong to, and a *range*, the class its objects belong to. It may also carry logical characteristics. Table 3 lists them for Meridian.

Property	Domain	Range	Characteristics
`works_in`	Person	Department	functional
`reports_to`	Person	Person	functional, transitive, irreflexive
`works_on`	Person	Project	
`leads`	Person	Project	subproperty of `works_on`
`uses_tech`	Project	Technology	
`delivered_for`	Project	Client	functional
`has_skill`	Person	Technology	
`located_in`	Department or Client	City	functional
`depends_on`	Technology	Technology	transitive, irreflexive
`mentors`	Person	Person	irreflexive

Table 3: Properties of the Meridian ontology. A functional property has at most one value per subject. The class hierarchy gives `Manager`, `Engineer` and others as subclasses of `Person`, and `Language`, `Database` and others as subclasses of `Technology`.

The class hierarchy has a practical consequence that surprises newcomers. The data state that Python is a `Language`. They never state that it is a `Technology`. A literal query for all technologies therefore returns nothing. The answer lies in the schema: `Language` is a subclass of `Technology`, so every language is a technology. Making that step automatic is what reasoning is for.

3.2 RDFS, OWL and SHACL

Three W3C standards cover the schema layer, and they do different jobs.

RDFS.
A small vocabulary: `rdfs:subClassOf`, `rdfs:subPropertyOf`, `rdfs:domain` and `rdfs:range`. It supports simple inference.

OWL 2.
A richer logic with cardinality, property characteristics such as transitivity, class intersection and union, and more [13]. It is more expressive and its reasoning is more expensive.

SHACL.
A language for *validating* data against shapes [14]. It checks; it does not infer.

The distinction between inferring and checking rests on how each treats missing information. RDFS and OWL work under the *open-world assumption*: a statement that is absent is unknown, not false. If an employee has no recorded department, a reasoner concludes nothing. SHACL works as a constraint checker over the data as they stand, so a required property that is absent is reported as a violation. Both are needed. Inference enriches the graph; validation protects it.

Watch Out

RDFS reads `domain` and `range` as inference rules and combines several domains as an intersection. The property `located_in` applies to departments *and* clients. Declared as an RDFS domain of both classes, it would make every client a department as well. Expressing

"either" needs OWL class unions, or an explicit check outside RDFS. The lab uses the second route, which is why constraints live in `ontology.json` and are checked by code.

3.3 Forward chaining

The simplest way to apply schema rules is *forward chaining*, also called materialisation. Rules are applied repeatedly to the graph, each pass adding the conclusions that follow, until a pass adds nothing. For RDFS two rules do most of the work:

1. **Subclass rule.** If x has type C and C is a subclass of D, then x has type D.
2. **Subproperty rule.** If $x\ p\ y$ holds and p is a subproperty of q, then $x\ q\ y$ holds.

On the Meridian graph the first rule turns 0 technologies into 37, and the second adds the missing project memberships: a project lead whose own `works_on` edge was not recorded now appears on the team. The number of `works_on` triples grows from 204 to 229, and the whole RDF graph from 1,825 triples to 2,007. Without the schema, a query for everyone on Project Atlas would silently omit its lead.

Practical Tip

Materialise inferred triples when queries are frequent and the data change slowly. Compute inference at query time, for example with property paths, when data change often or when the closure would be large. Transitive properties such as `reports_to` are the usual case for query-time evaluation.

3.4 Constraints as executable schema

The schema also defines what is wrong. A triple whose head is a technology and whose relation is `works_on` violates the domain of the property. Section 5.3 turns the constraints of Table 3 into a validator and measures how well it finds errors that were injected into a copy of the graph.

4 Querying

4.1 SPARQL

SPARQL is the W3C standard language for querying RDF [15]. A query is a pattern of triples with variables. The engine returns every assignment of values to variables that makes the pattern match the graph. The following query lists the projects of one person and, where known, their clients:

```
SELECT ?project ?client WHERE {
  ?p  rdfs:label "Priya Nair" .
  ?p  ex:works_on ?pr .
  ?pr rdfs:label ?project .
  OPTIONAL { ?pr ex:delivered_for ?c . ?c rdfs:label ?client }
}
```

The `OPTIONAL` block keeps a project in the result even when no client is recorded, much as a left join does in SQL. Aggregation works as in SQL. This query finds the technologies held as a skill by the most people:

```
SELECT ?tech (COUNT(DISTINCT ?p) AS ?people) WHERE {
```

```
  ?p ex:has_skill ?t .  ?t rdfs:label ?tech .
}
GROUP BY ?tech ORDER BY DESC(?people) LIMIT 5
```

On the Meridian data it returns Python (29 people), TypeScript and PostgreSQL (22 each), then Elasticsearch and scikit-learn (21 each).

Property paths

Questions of unknown depth need *property paths*. The operator + means one or more steps, so the whole management chain above a person is a single pattern:

```
SELECT ?boss WHERE {
  ex:per_009 ex:reports_to+ ?b .   ?b rdfs:label ?boss .
}
```

The equivalent in a relational database is a recursive common table expression. In a graph language the recursion is a property of the pattern and not something the author has to write.

4.2 Paths as the unit of multi-hop questions

Most questions worth asking of a knowledge graph are paths. "Which clients has Priya Nair worked for?" follows `works_on` and then `delivered_for`. "Which people have a skill in a technology used by Project Atlas?" follows `uses_tech` forward and then `has_skill` backward. The lab uses a compact notation for such paths, which is also the target of the question-parsing step in Section 8.

Step	Meaning	Example
`rel>`	follow the relation forward	`works_on>`
`rel<`	follow the relation backward	`has_skill<`
`rel+>`	follow it transitively	`reports_to+>`

Table 4: Path steps. Steps are separated by commas, so that the clients of a person's projects are `works_on>,delivered_for>`.

An executor for this language is about 40 lines of Python. It keeps, for each entity reached, the chain of triples that led there. That chain is the *evidence* for the answer, and Section 8 uses it to build explanations.

4.3 Why traversal is cheap

Graph databases usually store, for every node, direct references to its neighbours. Following an edge is then a pointer lookup, whose cost depends on the number of neighbours and not on the size of the whole graph. A relational join over large tables pays for index lookups at every step. For shallow queries the difference is small and well-tuned relational systems are competitive; it grows with the depth of the traversal. The practical advice is to measure on your own data before choosing a store for performance reasons, and to choose for the data model and the questions otherwise. **Hubs** are the usual exception: expanding all neighbours of a node like Python is expensive in any system.

5 Building the Graph

A graph is only as good as the pipeline that fills it. Figure 5 shows the stages used here. Structured records and free text enter on the left; a validated graph leaves on the right. Each stage has its own recall ceiling, and the losses multiply along a path, a theme that returns in Section 8.

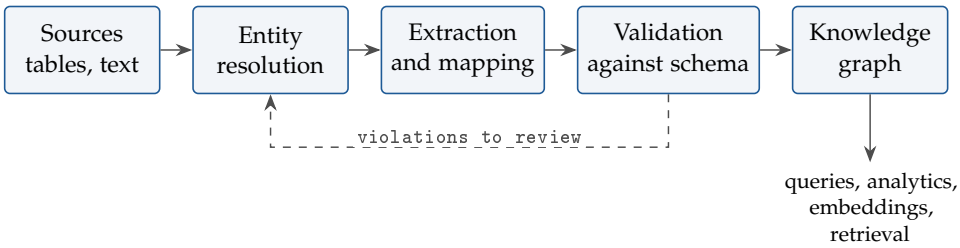


Figure 5: Construction pipeline. Violations found at validation go to a review queue and, once corrected, flow back to the stage that caused them.

5.1 Entity resolution

Before extraction or inference there must be one node per real entity. In the Meridian data, a person can appear in the HR system (`Elena Clarke`), in Jira (`Clarke, Elena`), in the wiki (`E. Clarke`) and in the directory (`ELENA CLARKE`). The file `raw_records.csv` holds 374 such records, 310 of people and 72 of clients, drawn from seven systems. Matching records to entities is called entity resolution, record linkage or deduplication, and is the subject of a long literature [16, 17, 18]. The standard pipeline has four stages.

Normalise. Remove superficial variation: fold case, strip punctuation, reorder `Last, First`, drop company suffixes such as *Ltd* and *Limited*, and map city codes to a canonical form.

Block. Comparing every pair of records is quadratic. Blocking compares a pair only when both records share a cheap key [18]. Here the key for a person is the pair (first initial, last initial). It survives typos in the middle of a name and works for records that carry only initials. For the 374 records this reduces 69,751 possible pairs to 676 candidates, a reduction of 99.0 percent, while keeping all 339 true pairs (pair completeness 1.000).

Compare. Each candidate pair receives a score from several signals. String similarity uses the Jaro–Winkler measure. Jaro similarity of two strings s_1 and s_2 is

$$\mathrm{sim}_J = \frac{1}{3}\left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m}\right), \tag{1}$$

where m is the number of matching characters and t is half the number of transpositions [19]. Winkler's variant adds a bonus for a shared prefix of length $\ell \le 4$, $\mathrm{sim}_{JW} = \mathrm{sim}_J + \ell p\,(1 - \mathrm{sim}_J)$ with $p = 0.1$ [20]. For people the score is a weighted sum

$$s = 0.35\,\mathrm{name}(f_1, f_2) + 0.35\,\mathrm{name}(l_1, l_2) + 0.20\,e + 0.10\,c, \tag{2}$$

where name is Jaro–Winkler similarity, or 1 if one side is an initial that matches the other, and 0 if the initials differ. The terms e (e-mail) and c (city) equal $+1$ when both values are present and equal, -1 when both are present and different, and 0 when either is missing. Treating a

mismatch as evidence against a merge, and a missing value as no evidence, is the central idea of the Fellegi–Sunter model [16]. The weights here are set by hand. That model supplies a principled way of estimating them from data.

Cluster. Pairwise decisions must become a partition. Connected components of the match graph are the simplest method, but a single ambiguous record can chain two real entities together. In the Meridian data the wiki record `Sneha S.` fits both Sneha Smith and Sarah Smith. A *cannot-link* constraint repairs this: a source system holds at most one record per person, so two clusters that both contain an HR record may not merge. Pairs are processed from highest to lowest score, and any merge that would place two records of one system in one cluster is refused.

Method	Precision	Recall	F1
Pairwise, threshold 0.66 (best F1)	0.991	0.994	0.993
Connected components	0.971	0.994	0.983
Components with cannot-link constraint	1.000	0.994	0.997

Table 5: Entity resolution, pairwise metrics over all 339 true pairs. Connected components produced two clusters that mixed several real entities; the constraint removed both.

Figure 6 shows the trade-off as the threshold moves. Raising it to the lowest value with no false pair at all (0.69) lowers recall from 0.994 to 0.976. The first pairs lost are client names with typos in the finance system, such as `Corvid argo` for Corvid Cargo.

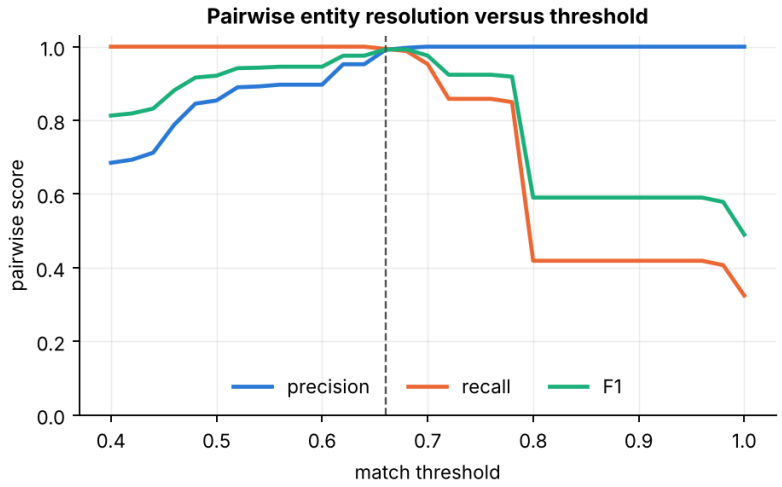


Figure 6: Precision, recall and F1 of pairwise matching against the threshold. The dashed line marks the best F1.

Key Idea

The two kinds of error do not cost the same. A false merge corrupts a node, because facts about two entities now sit on one. A missed merge only leaves a duplicate. Production systems usually fix a high-precision threshold and send the uncertain band to human review.

5.2 From text to triples

Much organisational knowledge lives in prose. A text-to-graph pipeline has two core steps: *entity linking*, which finds mentions and maps them to node identifiers, and *relation extraction*,

which decides which relation, if any, holds between the mentions in a sentence. Current systems often prompt a language model to produce triples. The method used here is classical and transparent so that every error can be traced, and the evaluation is the same whichever extractor is used.

The corpus has 739 sentences generated from true triples, so that `doc_gold_triples.csv` is an exact answer key of 859 triples. About a tenth of the sentences are traps: negations (*"X no longer works on Y"*), mentions without a relation (*"X attended the Y workshop"*) and filler.

Mention detection. A gazetteer maps surface forms to identifiers. The longest match at each position wins, so *Project Atlas* beats *Atlas*. Ambiguous forms such as `P. Nair`, which fits two people, are left out, because linking them would be a guess.

Relation patterns with type constraints. Each pattern pairs a trigger phrase with a relation and a direction. For a trigger, the head is the nearest preceding mention whose type fits the domain of the relation, and the tails are the following mentions that fit its range. Some triggers are reversed: *"X manages Y"* yields (Y, `reports_to`, X). Using the ontology as a filter is the most effective single device in the extractor. It removes most spurious candidates without any learning.

Scoring. Extracted triples are compared with the answer key sentence by sentence.

Extractor	Extracted	Correct	Precision	Recall	F1
Patterns and types	773	758	0.981	0.882	0.929
+ negation guard	758	758	1.000	0.882	0.938
+ complete gazetteer	821	821	1.000	0.956	0.977

Table 6: Triple extraction against the answer key (859 gold triples).

Reading the errors matters more than the score. The 15 false positives all come from the single cause visible in the trap sentences: *"Sarah Smith no longer works on Aurora"* matches the trigger *works on*. A negation guard removes them. The 101 false negatives are all *linking* failures: a mention was not found because the gazetteer lacks that surface form. Not one is a pattern failure. No amount of pattern tuning would repair them. Adding the remaining unambiguous aliases lifts recall from 0.882 to 0.956, and the 38 triples still missed all sit in sentences that contain an ambiguous form.

The graph extracted from text is accurate and incomplete. It holds 533 distinct triples, all of them correct, which is 40.0 percent of the 1,332 in the true graph. Text mentions only part of what the structured sources know, and Section 8 shows what that incompleteness does to question answering.

Watch Out

Extraction scores measured on generated sentences are optimistic. Real text has longer sentences, coreference, implicit relations and hedged claims. Treat these numbers as a demonstration of the method and its error analysis, and re-measure on a sample of your own documents. Distant supervision [21] is one way to obtain training labels without writing an answer key by hand.

5.3 Validating against the schema

Real graphs arrive dirty. The file `triples_dirty.csv` contains the clean edges plus 90 injected errors of six kinds, each labelled, so that detection can be measured. The validator applies the constraints of Table 3.

Constraint	Violation	Injected	Caught	Recall
domain	a technology that `works_on` a project	20	20	1.000
range	a project that `uses_tech` a person	30	30	1.000
functional	a person who `works_in` two departments	20	20	1.000
cycle	A reports to B and B reports to A	8	8	1.000
self loop	a person who `mentors` themselves	6	6	1.000
mutual	A mentors B and B mentors A	6	5	0.833

Table 7: Detection of injected errors. The validator flagged 90 rows in total, 89 of them correctly: precision 0.989 and recall 0.989.

Local rules (domain, range, self loop) look at one triple and are almost always right. Structural rules (functional violations, cycles, mutual relations) involve at least two edges, so they localise a problem without saying which edge is wrong. The validator breaks the tie with provenance, flagging the edge with the lower confidence. That works here because the injected edges carry lower confidence than the HR records they contradict. The one miss and the one false positive both come from the mutual-mentoring rule, where the two directions have similar confidence and the choice is close to a coin flip.

Practical Tip

In production run the same checks on every load, for example as SHACL shapes, and route violations to a reviewable queue. Do not repair silently. A violation is information about an upstream system.

6 Analysing the Graph

Once the graph exists, its structure answers questions that tables answer badly. This section covers four families of analysis: centrality, communities, impact analysis and rule mining. The first two use the *collaboration graph*: two people are linked if they work on a common project, and the edge weight is the number of shared projects. It has 117 people (the three executives have no projects), 454 edges and a single connected component.

6.1 Centrality

Centrality measures rank nodes by structural importance. Three are common.

- **Degree** counts neighbours. It is cheap and measures local activity.
- **Betweenness** counts the shortest paths that pass through a node,

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}, \tag{3}$$

  where σ_{st} is the number of shortest paths from s to t and $\sigma_{st}(v)$ is the number of those that pass through v [22]. It finds bridges between groups.

- **PageRank** scores a node by the scores of the nodes that point to it [23]. With damping factor d and n nodes,

$$\mathrm{PR}(v) = \frac{1-d}{n} + d \sum_{u \to v} \frac{\mathrm{PR}(u)}{\deg^{+}(u)}. \tag{4}$$

Betweenness is the most useful for organisational questions. The highest value in the collaboration graph belongs to a person in Product Engineering who works on four projects with 15 collaborators. People like this connect teams that an organisation chart shows as separate, and they are the ones whose absence would cut the most communication paths.

6.2 Communities

Community detection partitions a graph into densely connected groups. Louvain optimises *modularity* [24, 25],

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{5}$$

which compares the weight inside groups with what a random graph of the same degrees would give. It finds 8 communities. To judge them we compare the partition with the real departments using the adjusted Rand index [26], which is 0 for chance agreement and 1 for identical partitions, and normalised mutual information. The values are 0.60 and 0.71.

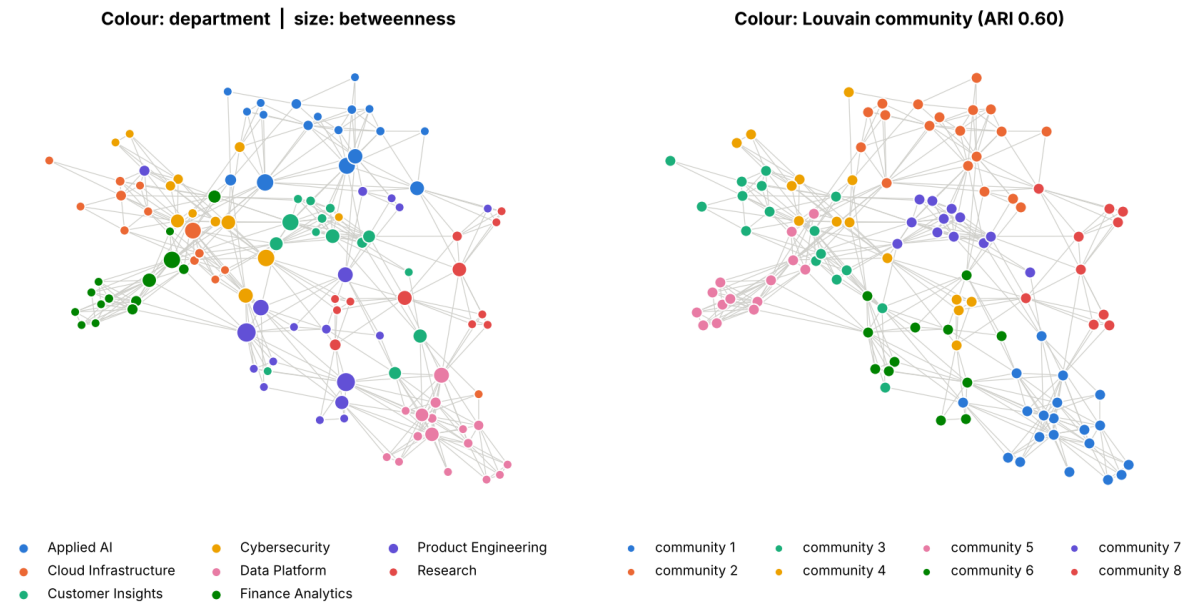


Figure 7: The collaboration graph. Left: colour shows department and size shows betweenness. Right: colour shows the Louvain community; the colours are arbitrary labels.

The communities follow the departments closely but not exactly. About 32 percent of the collaboration edges join people in different departments, which is the structure that a `GROUP BY department` hides.

6.3 Impact analysis

If a vulnerability is announced in a technology, which projects are affected? A project is affected if it uses that technology directly or uses anything that depends on it, directly or indirectly. This is a transitive query over the `depends_on` relation followed by a lookup of `uses_tech`.

Technology	Direct projects	Dependent technologies	Affected projects
Python	14	10	32
Java	3	7	26
SQL	8	1	13
Go	5	3	12
TypeScript	6	1	11

Table 8: Projects affected by a problem in each technology, out of 45. Java is used directly by three projects but affects 26, because Scala, Spark, Kafka, Spring Boot and others rest on it.

The gap between the direct and the affected count is the point of the analysis. A tabular report of "projects that use Java" would give 3. The boundary of the method is equally important: it sees only the dependencies that someone recorded. The graph says what the data state depends on what, never what really depends on what.

6.4 Rule mining

Rules turn regularities into explicit, inspectable knowledge. A rule has a body and a head. For example

$$\texttt{works_on}(x,z) \ \wedge \ \texttt{uses_tech}(z,y) \ \Rightarrow \ \texttt{has_skill}(x,y) \tag{6}$$

says that a person who works on a project that uses a technology probably has that skill. Three quantities describe a rule [27].

- **Support**: the number of pairs (x, y) for which body and head both hold.
- **Confidence**: support divided by the number of pairs for which the body holds.
- **Head coverage**: support divided by the number of facts with the head relation.

Matrix arithmetic makes the search cheap. If A_r is the adjacency matrix of relation r, the body $r_1(x,z) \wedge r_2(z,y)$ is the Boolean product $A_{r_1} A_{r_2}$, and an inverse relation is the transpose. The lab enumerates all bodies of length one and two for each of the ten head relations.

Rule	Support	Confidence	Head cov.
$\texttt{mentors}^{-1}$(x,z) ∧ works_in(z,y) ⇒ works_in(x,y)	16	1.000	0.133
reports_to(x,z) ∧ works_in(z,y) ⇒ works_in(x,y)	111	0.933	0.925
works_on(x,z) ∧ uses_tech(z,y) ⇒ has_skill(x,y)	344	0.472	0.639
leads(x,y) ⇒ works_on(x,y)	20	0.444	0.098
reports_to(x,z) ∧ has_skill(z,y) ⇒ has_skill(x,y)	153	0.284	0.284

Table 9: Selected rules among the 27 with support of at least 15. $^{-1}$ marks the inverse relation.

Three kinds of rule appear. The first two rows are *organisational regularities*: a person usually works in the same department as the person they report to, and every mentee works where the mentor does. The fourth row is a schema axiom rediscovered from data. Its confidence is only 0.44 because about half of the project leaders' `works_on` edges were never recorded, so the rule doubles as a missing-data detector. The third row is a *soft* rule. It is right about half the time, and applied to the 729 person and technology pairs for which its body holds, it proposes 385 `has_skill` facts that are not yet recorded. Those are candidates for review, not conclusions.

Watch Out

Confidence is computed under the closed-world assumption: facts that are not in the graph count as false. For a sparse relation such as `has_skill` this makes confidence

pessimistic, because some failures are simply unrecorded skills. AMIE corrects for this with a partial-completeness assumption [27].

7 Embeddings and Link Prediction

Rules are exact and readable, but they cannot weigh many weak signals at once. Embedding models take the opposite approach. They place every entity and relation in a vector space and train the vectors so that true triples score higher than false ones. The result predicts links that are not in the graph and gives every entity a compact numerical description. Surveys of the area are given in [33, 34].

7.1 Two classic models

TransE treats a relation as a translation [28]. For a true triple (h, r, t) the embeddings should satisfy $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ (Figure 8). The score is the negative distance,

$$f(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2. \tag{7}$$

Training minimises a margin ranking loss. For each true triple a corrupted triple (h', r, t') is built by replacing the head or the tail with a random entity, and the loss is

$$\mathcal{L} = \sum_{(h,r,t)} \max\bigl(0,\ \gamma + \|\mathbf{h} + \mathbf{r} - \mathbf{t}\| - \|\mathbf{h}' + \mathbf{r} - \mathbf{t}'\|\bigr), \tag{8}$$

with margin γ. Entity vectors are renormalised to unit length after every epoch, which stops the model from reducing the loss by shrinking all vectors.

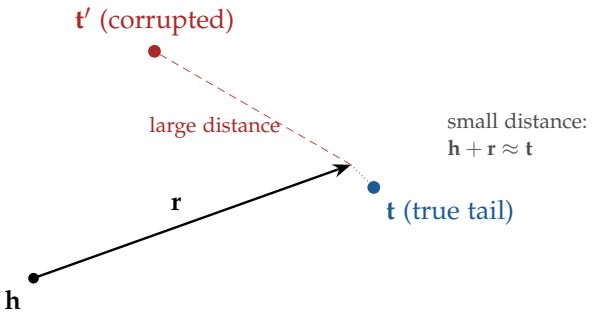


Figure 8: TransE geometry. Adding the relation vector to the head should land near the true tail and far from a corrupted one.

DistMult scores a triple with a three-way product [29],

$$f(h, r, t) = \sum_i h_i \, r_i \, t_i, \tag{9}$$

and is trained with a logistic loss against sampled negatives. The score is symmetric in h and t, so DistMult cannot tell direction. ComplEx repairs this with complex-valued vectors [30], RotatE models relations as rotations in the complex plane [31], and relational graph convolutional networks add message passing over the neighbourhood [32]. The lab implements TransE and DistMult from scratch in NumPy, with hand-derived gradients, because the mechanics are clearer without a framework.

7.2 Evaluation protocol

The 1,332 triples are split into 1,066 for training, 132 for validation and 133 for testing, keeping only test triples whose entities appear in training. For each test triple the tail is hidden and every entity is ranked as a candidate, then the same is done for the head. Four conventions matter.

- **Filtered ranking.** Candidates that are known true answers are removed, so a model is not penalised for ranking another correct answer above the held-out one.
- **Mean reciprocal rank** (MRR) is the average of 1/rank of the true entity, and **Hits@*k*** is the share of cases with the true entity in the top k.
- **Baselines.** A random scorer and a popularity scorer, which ranks entities by how often they occur with the relation in training. A model that cannot beat popularity has learned nothing about structure.
- **Typed candidates.** An optional step that ranks only entities satisfying the domain or range of the relation. It feeds the ontology of Section 3 back into the evaluation.

7.3 Results

Each learned model was trained from three random initialisations. TransE used dimension 50, margin 1.0, learning rate 0.05 and 200 epochs. DistMult used dimension 50, five negatives per positive, Adagrad with rate 0.1, L2 weight 0.001 and 100 epochs. Table 10 gives means over the three runs.

Candidates	Model	MRR	Hits@1	Hits@3	Hits@10	Mean rank
All entities	Random	0.018	0.004	0.004	0.019	122.8
	Popularity	0.111	0.041	0.090	0.263	51.0
	DistMult	0.229	0.127	0.234	0.454	36.3
	TransE	**0.314**	**0.163**	**0.360**	**0.664**	**12.5**
Typed	Random	0.098	0.030	0.075	0.222	34.4
	Popularity	0.113	0.041	0.090	0.263	37.4
	DistMult	0.278	0.149	0.293	0.550	16.6
	TransE	**0.314**	**0.163**	**0.362**	**0.664**	**12.4**

Table 10: Filtered link prediction on 133 held-out triples, 266 ranked cases. Standard deviation of MRR over three seeds: 0.007 for TransE and 0.013 to 0.016 for DistMult.

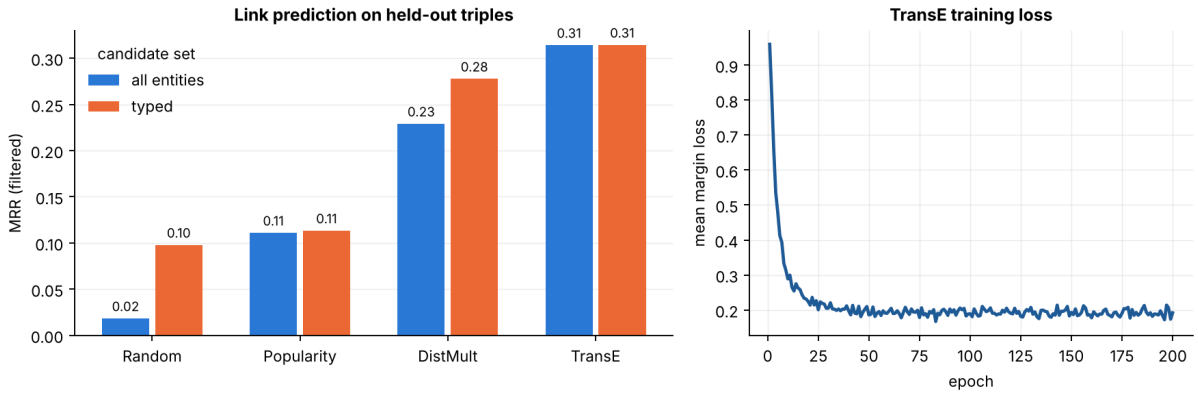


Figure 9: Left: filtered MRR for each model and candidate set. Right: TransE training loss, which settles within about 30 epochs.

Four observations follow from Table 10.

1. Both learned models beat popularity clearly, so they capture structure beyond "common entities are common answers".
2. Typing the candidates helps the weaker model most. DistMult wastes much of its ranking on entities of the wrong kind, while TransE has already learned the types, so its score does not move.
3. Typing lifts even the random scorer, from an MRR of 0.02 to about 0.10. Models must therefore be compared within one candidate setting, never across settings.
4. TransE leads on this graph. That ordering reflects the size and shape of a small graph with many one-to-many relations, and it should not be read as a general verdict. Comparative studies on large benchmarks show that careful tuning changes the ranking of models considerably [35].

7.4 Which relations are learnable

A per-relation breakdown for one TransE run shows where the structure is.

Relation	Ranked cases	MRR	Hits@10
`works_in`	16	0.958	1.000
`reports_to`	20	0.505	1.000
`uses_tech`	38	0.328	0.789
`works_on`	38	0.303	0.737
`has_skill`	122	0.209	0.525
`delivered_for`	10	0.153	0.400

Table 11: Per-relation results for TransE with typed candidates (selected relations). Relations with few distinct values per head, such as `works_in`, are easy; relations with many plausible tails, such as `has_skill`, are hard.

7.5 What the space looks like

Projecting the entity vectors to two dimensions with PCA shows that the model has recovered the schema without being shown it. Entities of the same type form separate clusters (Figure 10, left). Differences within a type are smaller than those between types, so for people a second projection is fitted on people alone (right). A number backs the picture: for each person, 99.3 percent of their five nearest neighbours by cosine similarity work in the same department, against 12.8 percent expected by chance.

Key Idea

Embeddings and rules complement each other. Rules explain and are exact where they apply; embeddings generalise from many weak signals. A good workflow proposes candidate facts with embeddings, filters them with ontology constraints, and sends the survivors to human review.

Watch Out

Link prediction scores are not probabilities, and the closed-world evaluation above penalises a correct new fact that happens to be unrecorded. Treat the metrics as a way of comparing models, not as accuracy. Tune on the validation set and touch the test set once: on this graph, a configuration chosen on 132 validation cases scored no better on test than the untuned default, which is within seed noise.

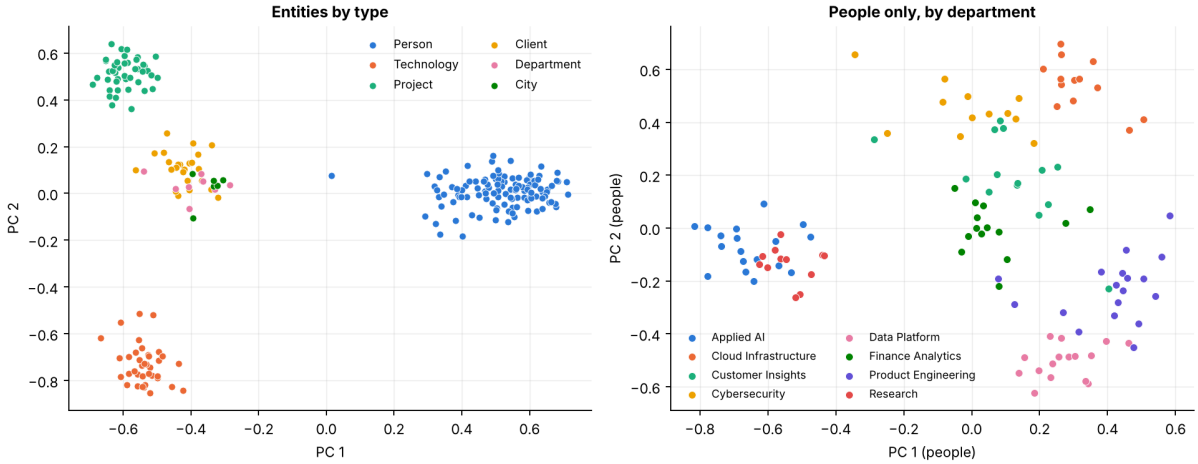


Figure 10: TransE entity embeddings projected by PCA. Left: all entities coloured by type. Right: people only, coloured by department.

8 Knowledge Graphs and Language Models

8.1 Where retrieval falls short

Retrieval-augmented generation (RAG) gives a language model passages that were retrieved for the question, so that the answer rests on documents and not only on what the model memorised [36]. Passages are usually chosen by text similarity. That works when the answer sits in one or two passages. It works poorly when the answer must be *assembled*: "which clients has this person worked for" needs the person's projects from one place and each project's client from another. Similarity search has no notion of joining facts, of counting, or of showing the chain that produced an answer.

A knowledge graph supplies exactly those three things. It retrieves by following relations, it joins and aggregates exactly, and it returns the triples used, which serve as an explanation. Two related lines of work should be kept apart. Microsoft's GraphRAG builds a graph *from text* with a language model, detects communities in it and pre-summarises them, which is aimed at global questions about a corpus [37]. The experiment here uses a graph that already exists. A broader roadmap of how language models and knowledge graphs can support each other is given in [38].

8.2 The experiment

The file `questions.csv` holds 60 questions of one to three hops, built from 12 templates with exact gold answers. Three systems answer them.

System	Knowledge source	How it answers
A. Text retrieval	the 739 sentences	TF-IDF retrieves the five most similar sentences; the answer is the set of entities of the right type mentioned in them
B. Graph from text	triples extracted in Section 5.2	parse the question into a path, then traverse the extracted graph
C. Curated graph	`triples.csv` with RDFS inference	parse the question, then traverse the full graph

Table 12: The three systems compared.

System A has a deliberately generous reader. It picks out the entities of the right type perfectly, which a real pipeline could not guarantee, so its numbers are an upper estimate of what retrieval alone achieves. Systems B and C turn the question into a path of the kind described in Section 4.2. Because the questions follow templates, a regular-expression parser handles them with perfect accuracy; a production system would use a language model for this step, or let it write the graph query directly. The score is the F1 of the returned answer set against the gold set.

8.3 Results

Questions	Number	A. Text	B. Graph from text	C. Curated graph
1 hop	30	0.467	0.690	1.000
2 hops	25	0.112	0.207	1.000
3 hops	5	0.000	0.260	1.000
All	60	0.280	0.453	1.000
Mean recall	60	0.316	0.405	1.000

Table 13: Mean answer F1 by number of hops, and mean recall over all questions.

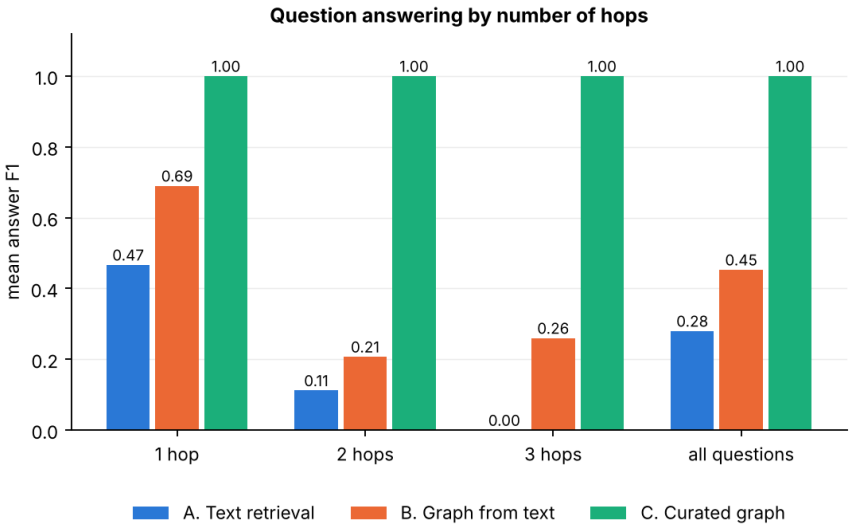


Figure 11: Answer F1 by number of hops for the three systems.

The curated graph answers every question exactly. That is expected, because the gold answers were computed from it, so the comparison measures what the other two sources lose, not the quality of C. Three things stand out.

1. **Text retrieval collapses as questions need joins.** It reaches 0.47 on one-hop questions and 0.11 on two-hop questions. A single sentence rarely contains both links of a chain, and the second link is usually in a sentence that does not resemble the question.
2. **The graph built from text is limited by extraction recall.** Every sentence the extractor misses is a missing edge, and missing edges compound along a path. Its recall is 0.41, close to the 40 percent of the true graph that the text contains. Only five questions have three hops, so the apparent rise from two hops to three is noise.
3. **The schema contributes.** Without the subproperty rule, the 20 questions whose paths cross `works_on` score 0.953 instead of 1.000, because project leaders disappear from the answers.

8.4 Evidence-guided context

A graph-based RAG system passes the evidence to the language model. The naive way hands over the whole neighbourhood of the question's entity. The guided way hands over only the triples that lie on answer paths. For "Which clients has Priya Nair worked for?" the two-hop neighbourhood contains 235 facts. The guided evidence contains 6:

```
Answer using only the facts below.

Facts:
- Priya Nair  --works_on-->  Project Orion
- Priya Nair  --works_on-->  Project Lyra
- Priya Nair  --works_on-->  Project Apollo
- Project Orion  --delivered_for-->  Orbis Logistics
- Project Lyra  --delivered_for-->  Maple and Mint Stores
- Project Apollo  --delivered_for-->  Corvid Cargo

Question: Which clients has Priya Nair worked for?
```

The short context is cheaper, leaves less room for distraction, and can be checked by a person. The neighbourhood is larger because it includes hubs: expanding the skills of every colleague, for instance, pulls in Python and its dozens of edges.

Key Idea

The graph does two jobs in a RAG system. It *finds* relevant facts by following relations that a text embedding cannot see, and it *explains* the answer, because each answer comes with the chain of triples that produced it.

8.5 Failure modes

Watch Out

Graph retrieval fails silently when the graph is incomplete. A missing edge produces a confident but partial answer, with no sign that anything is missing. System B above shows this: it answers every question, and on average finds 41 percent of the right answers.

Other failures deserve a test each.

- **Wrong parse.** The question is mapped to the wrong path or the wrong start entity. Log the parsed path and review a sample.
- **Stale facts.** An edge that was true last year is returned today. Store a validity interval and filter on it.
- **Ambiguous names.** A mention links to the wrong node. Resolve against the whole context, not the string.
- **Over-trust.** A language model asked to answer from facts may still fill gaps from memory. Instruct it to say when the facts are insufficient, and test that it does.

Report coverage alongside accuracy, and keep a text-retrieval fallback for questions the graph cannot answer.

9 Engineering Practice

The previous sections showed methods. This section collects the decisions that decide whether a knowledge graph survives contact with an organisation. The lessons agree with reports from industry-scale projects [6].

9.1 Decisions to make early

Identifiers. Give every entity an opaque, stable identifier and treat labels as attributes. A label changes when a person marries or a product is renamed; an identifier must not. The Meridian graph uses prefixes such as `per_` and `prj_` only for readability.

Provenance and time. Every edge in the lab carries `source`, `confidence` and `valid_from`. Those three columns made the repair rules of Section 5.3 possible and would make audit possible. They are cheap to record at load time and expensive or impossible to reconstruct later. If the facts can end as well as begin (a person leaves a project), store an interval and not just a start.

Schema ownership. Decide who may change the ontology and how changes are versioned. Schema changes are cheap in a graph and costly for every consumer that assumed the old one.

Where the truth lives. Choose a source of record for each relation. The graph integrates; it should not become a second, unsynchronised copy of the HR system.

9.2 Quality

Measure quality continuously instead of once. Four dimensions are practical to track.

Dimension	Question	Example measure in this note
Accuracy	Are stated facts true?	precision of extraction (1.000 after the negation guard)
Completeness	Are true facts present?	share of the true graph recovered from text (0.40)
Consistency	Do facts obey the schema?	constraint violations per load (90 found, 89 correct)
Timeliness	Are facts current?	share of edges past their validity interval

Table 14: Quality dimensions with the measures used in the lab.

9.3 Common pitfalls

9.4 People and privacy

A graph that links people to projects, skills and managers is personal data in most jurisdictions. Decide which relations may be shown to whom before the graph is queried widely. In the Meridian data the skills survey is the sensitive source and also the least reliable, with a mean confidence of 0.77, which is a reason to show derived skill suggestions as suggestions to the person concerned and not as facts in a directory. Keep a person in the loop for any decision that affects someone.

Pitfall	Symptom	Remedy
Merging too eagerly	facts about two people on one node	high-precision threshold; cannot-link constraints; review queue
Trusting one metric	high F1 that hides a systematic error	read the errors; split them by cause (Section 5.2)
Evaluating on the test set during tuning	optimistic results that vanish on new data	validation set for choices; test once
Comparing across candidate sets	a weak model looks strong	fix the evaluation protocol first (Section 7.3)
Silent incompleteness	confident partial answers	report coverage; keep a text fallback
Treating inference as truth	derived facts that are plausible but wrong	mark inferred edges; keep them separable from asserted ones
Over-modelling	an ontology no one can maintain	start from the questions users ask; add classes only when a query or a constraint needs them

Table 15: Pitfalls seen repeatedly, with remedies.

9.5 A staged way to start

1. List the ten questions users most want answered, and write them as paths over entities and relations.
2. Model only the entities and relations those questions need, and load them from one or two reliable sources.
3. Add validation and provenance before adding sources.
4. Add entity resolution when the second source arrives, because that is when duplicates appear.
5. Add text extraction and learning only after the structured core answers the original questions.

10 Using the Lab Notebook

The notebook `knowledge_graphs_lab.ipynb` is the hands-on half of this note. It has ten labs, each a short explanation followed by code that runs on the files in `data/`, a few questions to answer, and worked solutions in an appendix. Every cell is already executed, so it can be read without running anything.

10.1 Map from the note to the labs

10.2 Running it

The notebook needs Python 3.10 or later with `pandas`, `numpy`, `scipy`, `scikit-learn`, `networkx`, `rdflib`, `matplotlib` and `python-louvain` (or a recent NetworkX with `louvain_communities`). Install them, open the notebook from the project folder, and run the cells in order. The whole notebook runs in about twenty seconds. The data are generated with a fixed seed and are identical on every run; to regenerate them, run `python scripts/generate_data.py`.

```
kg_tutorial/
  knowledge_graphs_lab.ipynb
  data/        entities, triples, ontology, aliases, gazetteer,
               documents, sentences, gold triples, raw records,
```

Lab	Note section	What you do
1	1	Turn tables into triples, count relations and degrees, draw a subgraph
2	2	Load the graph into NetworkX as a property graph and traverse it
3	3, 4	Write SPARQL with rdflib; apply RDFS rules by forward chaining
4	3, 5	Validate the dirty triples against the ontology and score detection
5	5	Resolve duplicate records with blocking, Jaro–Winkler and a cannot-link constraint
6	5	Extract triples from text, then read the errors
7	6	Centrality, Louvain communities, impact analysis
8	6	Mine rules with sparse matrices
9	7	Train TransE and DistMult in NumPy and evaluate them
10	8	Compare text retrieval with graph traversal and build a RAG prompt

Table 16: Which lab goes with which section.

```
                dirty triples, questions, README.md
figures/        images used in the note, results.json
scripts/        generate_data.py, notebook source and builder
note/           LaTeX source of this note
```

The file `data/README.md` is a data dictionary. It lists every column, the planted structure and the difficulties built into the data on purpose.

Practical Tip

Work the labs with the questions closed. Write your answer first, then open the worked solution. The difference between what you expected and what the output shows is where most of the learning is.

10.3 Why the data are synthetic

Meridian Analytics is invented. Using generated data has three advantages for teaching. The answer key is exact, so every method can be scored. The difficulty is placed deliberately (typos, ambiguous names, negations, injected errors), so each method meets the problem it exists to solve. And nothing in it is personal data. The cost is that results are cleaner than real ones, which is why the text points out where a real project would do worse.

11 Exercises

The first group needs only the note. The second group asks for small changes to the lab code.

11.1 Concepts

1. A table has columns `employee`, `manager` and `department`. Write its rows as triples, and say which column becomes an edge to an entity and which would be better kept as a literal. *Hint: ask whether you would ever want to list everything linked to the value.*

2. Give one query that is easy in SPARQL or Cypher and awkward in SQL, and one that is the other way round. *Hint: think about path length that is not known in advance, and about aggregation over a fixed schema.*
3. In RDFS, `leads` is a subproperty of `works_on`. A person leads project P. What can be inferred, and what can *not* be inferred? *Hint: the rule runs in one direction.*
4. Why does RDFS never report a domain violation, while SHACL does? *Hint: compare what each does with a triple that disagrees with the schema.*
5. Explain why a missed merge in entity resolution is usually cheaper than a false merge.
6. A relation extractor has precision 0.98 and recall 0.60. A colleague says it needs a better model. What would you check first? *Hint: where do the 101 false negatives of Section 5.2 come from?*
7. Why can betweenness centrality identify people who matter to an organisation but who have a modest title?
8. A rule has confidence 0.47 and head coverage 0.64. Describe in words what each number says, and say whether you would act on the rule.
9. TransE cannot represent a symmetric relation well, and DistMult cannot represent an antisymmetric one. Explain both, using the scores in Section 7.1.
10. The text-only retrieval system scores 0.11 on two-hop questions. Give two reasons, other than the quality of the retriever.

11.2 Lab extensions

11. Add the relation `mentors` to the departmental rule in Lab 8 and report the new support and confidence.
12. In Lab 5, change the blocking key to the full last name. Measure the reduction ratio and pair completeness, and say which records are now lost.
13. In Lab 6, add a pattern for a phrase that appears in the sentences but is not yet covered. Check precision and recall before and after.
14. In Lab 9, train TransE with margin 0.5 and with margin 2.0. Plot validation MRR against margin, then report the test MRR of the best value once only.
15. In Lab 10, write a parser rule for one new question template and add its gold answers. Which system gains?
16. Choose a relation that is symmetric in your own domain. Define a SHACL-style check that flags a one-sided statement, and implement it on the dirty triples.

References

[1] A. Singhal. Introducing the Knowledge Graph: things, not strings. Google Official Blog, 2012.

[2] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, Z. Ives. DBpedia: a nucleus for a web of open data. In *ISWC*, 2007.

[3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor. Freebase: a collaboratively created graph database for structuring human knowledge. In *SIGMOD*, 2008.

[4] F. Suchanek, G. Kasneci, G. Weikum. YAGO: a core of semantic knowledge. In *WWW*, 2007.

[5] D. Vrandečić, M. Krötzsch. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.

[6] N. Noy, Y. Gao, A. Jain, A. Narayanan, A. Patterson, J. Taylor. Industry-scale knowledge graphs: lessons and challenges. *Communications of the ACM*, 62(8):36–43, 2019.

[7] A. Hogan et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):71, 2021.

[8] R. Cyganiak, D. Wood, M. Lanthaler (eds). RDF 1.1 Concepts and Abstract Syntax. W3C Recommendation, 2014.

[9] T. Berners-Lee, J. Hendler, O. Lassila. The Semantic Web. *Scientific American*, 284(5):34–43, 2001.

[10] R. Angles, C. Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1, 2008.

[11] N. Francis et al. Cypher: an evolving query language for property graphs. In *SIGMOD*, 2018.

[12] T. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[13] W3C OWL Working Group. OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation, 2012.

[14] H. Knublauch, D. Kontokostas (eds). Shapes Constraint Language (SHACL). W3C Recommendation, 2017.

[15] S. Harris, A. Seaborne (eds). SPARQL 1.1 Query Language. W3C Recommendation, 2013.

[16] I. Fellegi, A. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

[17] A. Elmagarmid, P. Ipeirotis, V. Verykios. Duplicate record detection: a survey. *IEEE TKDE*, 19(1):1–16, 2007.

[18] P. Christen. *Data Matching*. Springer, 2012.

[19] M. Jaro. Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida. *JASA*, 84(406):414–420, 1989.

[20] W. Winkler. String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage. In *Proc. Section on Survey Research Methods, ASA*, 1990.

[21] M. Mintz, S. Bills, R. Snow, D. Jurafsky. Distant supervision for relation extraction without labeled data. In *ACL-IJCNLP*, 2009.

[22] U. Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.

[23] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank citation ranking: bringing order to the web. Stanford InfoLab Technical Report, 1999.

[24] V. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008, 2008.

[25] M. Newman. Modularity and community structure in networks. *PNAS*, 103(23):8577–8582, 2006.

[26] L. Hubert, P. Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.

[27] L. Galárraga, C. Teflioudi, K. Hose, F. Suchanek. AMIE: association rule mining under incomplete evidence in ontological knowledge bases. In *WWW*, 2013.

[28] A. Bordes, N. Usunier, A. García-Durán, J. Weston, O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.

[29] B. Yang, W. Yih, X. He, J. Gao, L. Deng. Embedding entities and relations for learning and inference in knowledge bases. In *ICLR*, 2015.

[30] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, G. Bouchard. Complex embeddings for simple link prediction. In *ICML*, 2016.

[31] Z. Sun, Z.-H. Deng, J.-Y. Nie, J. Tang. RotatE: knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.

[32] M. Schlichtkrull, T. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling. Modeling relational data with graph convolutional networks. In *ESWC*, 2018.

[33] M. Nickel, K. Murphy, V. Tresp, E. Gabrilovich. A review of relational machine learning for knowledge graphs. *Proceedings of the IEEE*, 104(1):11–33, 2016.

[34] S. Ji, S. Pan, E. Cambria, P. Marttinen, P. Yu. A survey on knowledge graphs: representation, acquisition, and applications. *IEEE TNNLS*, 33(2):494–514, 2022.

[35] A. Rossi, D. Barbosa, D. Firmani, A. Matinata, P. Merialdo. Knowledge graph embedding for link prediction: a comparative analysis. *ACM TKDD*, 15(2):14, 2021.

[36] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[37] D. Edge et al. From local to global: a graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024.

[38] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu. Unifying large language models and knowledge graphs: a roadmap. *IEEE TKDE*, 36(7):3580–3599, 2024.

Dr. Anish Roychowdhury | Generative AI Architect & Educator